



Database Management System (303105203)

Unit – 9: PL/SQL

Dr. Dhanraj Verma,
Professor, CSE-AIML



PL/SQL Concepts:

- View
- PL/SQL Block
- Cursor
- Trigger
- Stored Procedure
- StoreFunction



Views

- The view is a virtual table. It does not physically exist. Rather, it is created by a query joining one or more tables.
- A view contains rows and columns, just like a real table
- The fields in a view are fields from one or more real tables in the database

Creating an SQL VIEW

Syntax:

```
CREATE VIEW view_name AS  
SELECT column_name(s)  
FROM table_name  
WHERE condition;
```

Views

View Creation – Example

1. CREATE VIEW transaction_view

AS SELECT acc_no, tr_date, amt from transaction;

2. Create view account_view

AS SELECT a.acc_no, a.balance, l.loan_amt, l.interest_rate from account a, loan l

where a.acc_no=l.acc_no and a.city='Ahmedabad';

The above views would create a virtual table based on the result set of the select statement. To show created views:

1. SELECT * FROM transaction_view;

2. SELECT * FROM account_view;

Views

Updating View

You can modify the definition of a VIEW without dropping it by using the following syntax:

```
CREATE OR REPLACE VIEW view_name  
AS SELECT columns  
FROM table  
WHERE predicates;
```

1. Create or Replace view account_view

```
AS SELECT a.acc_no,a.balance,l.loan_amt,l.interest_rate, l.remaining_loan from  
account a, loan l where a.acc_no=l.acc_no and a.city='Mehsana';
```

Views

Deleting View

The syntax for dropping a VIEW :

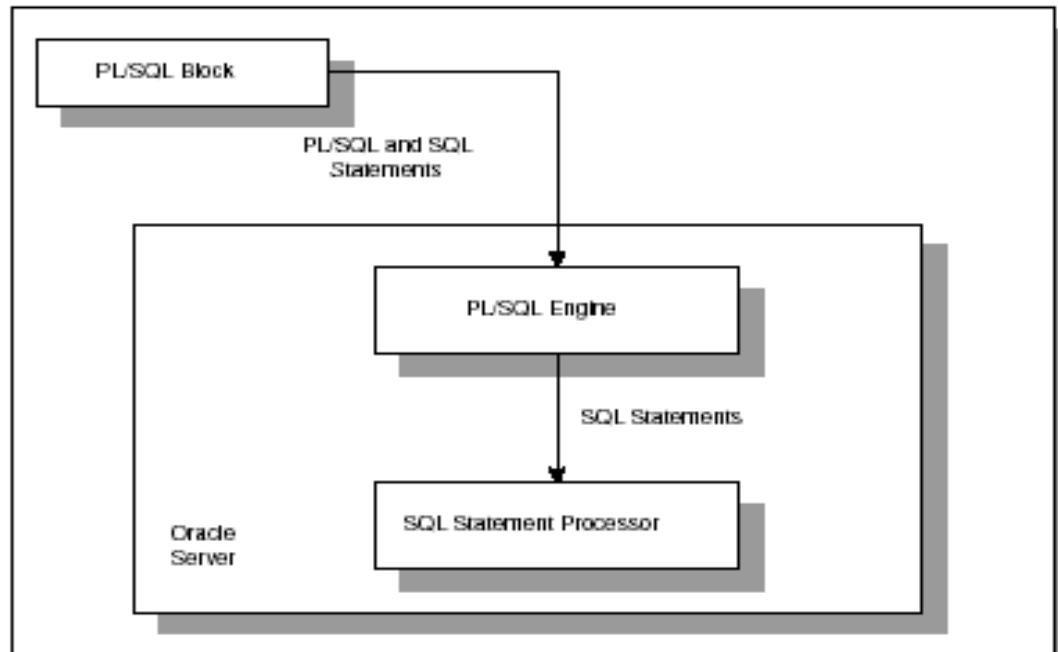
```
DROP VIEW view_name;
```

View Drop - Example

```
DROP VIEW transaction_view;
```

PL/SQL

Procedural Language – SQL: An extension to SQL with design features of programming languages (procedural and object oriented)





PL/SQL

PL/SQL BLOCK STRUCTURE

DECLARE (optional)

- variable declarations

BEGIN (required)

- SQL statements
- PL/SQL statements or sub-blocks

EXCEPTION (optional)

- actions to perform when errors occur

END; (required)

PL/SQL

- There are 2 types of Datatypes:
 1. Scalar (Char, Varchar2, Date and Number)
 2. Composite (%rowtype)
- All Variables required to be used in the program must be declared before their use. Variable names cannot be table, column and keyword names.
- Assignment Operator is denoted as “:=” and each statement of PL/SQL ends with “;”.
- Dbms_Output.Put_Line() is used for printing Output.
- &Variable_Name :Variable Name is use to take input from the user.

PL/SQL

➤ Conditional Statements

1. IF ...THEN...ELSE

```
IF <condition> THEN  
    <Code>  
ELSEIF <condition> THEN  
    <Code>  
ELSE  
    <Code>  
ENDIF;
```

2. CASE...

```
CASE  
WHEN VARIABLE = VALUE THEN  
    <Code>  
WHEN VARIABLE = VALUE THEN  
    <Code>  
ELSE  
    <Code>  
END CASE;
```

Types of Loops

1. Simple Loop

Loop

<Code>

Exit when <Cond>

End Loop;

2. While Loop

While <Condition>

Loop

<Code>

End Loop;

3. For Loop

FOR <Variable> IN <Min> .. <Max>

Loop

<Code>

End Loop;

4. For Loop

FOR <Variable> IN REVERSE <Min> .. <Max>

Loop

<Code>

End Loop;

PL/SQL Examples : Write a PL/SQL block to add 2 numbers

```
declare
x number(5);
y number(5);
z number(7);
begin
  x:=10;
  y:=20;
  z:=x+y;
  DBMS_OUTPUT.PUT_LINE('Sum is '||z);
end;
```

Parul

PL/SQL Examples

Write a PL/SQL block to find maximum of 2 numbers

```
declare  
x number(5);  
y number(5);  
begin  
  x:=10;  
  y:=20;
```

PU

if $x > y$ then

 DBMS_OUTPUT.PUT_LINE('x is greater than y');

else

 DBMS_OUTPUT.PUT_LINE(' y is greater than x');

end if;

end;

PL/SQL Examples:

Write a PL/SQL block to find area of rectangle, square and circle

```
declare
    area_c  number(6, 2) ;
    radius number(1) := 3 ;
    area_r number(6) ;
    area_s number(6) ;
    length number(2) := 5 ;
    width  number(2) := 4 ;
    pi constant number(3, 2) := 3.14;
```

PL/SQL Examples:

begin

```
area_c := pi * radius * radius;
```

```
DBMS_OUTPUT.PUT_LINE('Circle area = ' || area_c);
```

```
area_r := length * width;
```

```
DBMS_OUTPUT.PUT_LINE('Rectangle area = ' || area_r);
```

```
area_s := length * length;
```

```
DBMS_OUTPUT.PUT_LINE('Square area = ' || area_s);
```

end;

PL/SQL Examples

Write a PL/SQL block to print the sum of Numbers from 1 to 100

declare

a number;

sum1 number :=0;

begin

a:=1;

loop

sum1:=sum1+a;

a:=a+1;

exit when (a>100);

end loop;

DBMS_OUTPUT.PUT_LINE('sum of 1 to 100 is '||sum1);

end;

PL/SQL Examples

Write a PL/SQL program for inserting even numbers in EVEN table and odd number in ODD table from number 1 to 50.

```
begin
for i in 1..50
loop
if mod(i,2) = 0 then
insert into EVEN values (i);
end if;
end loop;
for i in 1..50
loop
```

```
if mod(i,2) != 0 then
insert into ODD values (i);
end if;
end loop;
end;
```

Cursor

- A cursor is a private set of records.
- A cursor is a temporary work area created in the system memory when a SQL statement is executed. A cursor contains information on a select statement and the rows of data accessed by it.
- This temporary work area is used to store the data retrieved from the database, and manipulate this data. A cursor can hold more than one row, but can process only one row at a time. The set of rows the cursor holds is called the *active set*.

There are 2 types of Cursors:

1. Implicit Cursor: Created automatically for every query in SQL.
2. Explicit Cursor: Created manually by the programmer.



Trigger

A trigger is a block structure which is fired when a DML statements like Insert, Delete, Update is executed on a database table. A trigger is triggered automatically when an associated DML statement is executed.

Syntax of Triggers

```
CREATE [OR REPLACE ] TRIGGER trigger_name  
{BEFORE | AFTER}  
{INSERT [OR] | UPDATE [OR] | DELETE}  
[OF col_name]  
ON table_name  
[REFERENCING OLD AS o NEW AS n]  
[FOR EACH ROW]  
WHEN (condition);
```

Trigger

Creating Trigger

1.

```
CREATE TRIGGER deleted_details  
BEFORE  
DELETE on loan  
FOR EACH ROW
```

2.

```
CREATE TRIGGER inserted_details  
after  
insert on transaction  
FOR EACH ROW
```

Trigger

Creating Trigger

3.

```
CREATE TRIGGER updated_details  
BEFORE  
UPDATE on account  
FOR EACH ROW when name='Patel Hiren';
```

Deleting Trigger

```
Drop trigger inserted_details on transaction;
```

Trigger

DROP TRIGGER *trigger_name* ON *tbl_name*;

- The syntax for a dropping a Trigger is:
- The syntax for a disabling a Trigger is:

ALTER TRIGGER *trigger_name* DISABLE;

- The syntax for a enabling a Trigger is:

ALTER TRIGGER *trigger_name* ENABLE;

Stored Procedure OR Stored Programs

- Can receive multiple input parameters and return multiple output values (or no output values) and perform DML Commands
- **Stored** PL/SQL program units are program units that other PL/SQL program can reference and that other database users can use and execute.
- A **program unit** is a self-contained group of program statements that can be used within a large program

Stored Procedure OR Stored Programs

```
CREATE [OR REPLACE] PROCEDURE procedure_name [ (parameter mod datatype  
[parameter]) ]  
IS  
    [local variable declaration_section]  
BEGIN  
    executable_section  
[EXCEPTION  
    exception_section]  
END [procedure_name];
```

Header
Section

body

Exception
section

Stored Procedure OR Stored Programs

- Create or replace: instructs the DBMS to create the procedure if it does not exist, otherwise; it replaces the existing one.
- The header section defines:
 1. The procedure **name** (name should be unique)
 2. The **parameters** that the procedure receives or delivers (IN, OUT, IN OUT)
 3. **IS** keyword: follows the parameters list
 4. The **local** procedure variables
- OR REPLACE clause is optional. **But** if omitted and a procedure exists with the same name, an **error** occurs

Stored Procedure OR Stored Programs

- Parameter **name**, **mod** and **data type** are enclosed in parentheses, each is separated by a comma.
- Parameter **mod**: describes how the procedure change the value. It can be:
 1. **IN**: passed parameter is a Read only value. Cannot be changed by the procedure
 2. **OUT**: write-only value. Always comes on the left side of an assignment statement
 3. **IN OUT**: its value can be changed

Function

- Functions can accept one, many, or no parameters, but a function must have a return clause in the executable section of the function.
- The datatype of the return value must be declared in the header of the function.
- A function is similar to a procedure, except that it returns a single value.

- The syntax for creating a function is as follows:

```
CREATE [OR REPLACE] FUNCTION function_name  
    (parameter list)  
    RETURN datatype  
IS  
BEGIN  
    <body>  
    RETURN (return_value);  
END;
```



Function

EXAMPLE:

```
Create or replace function account_det
(account_num varchar2)
RETURN varchar2
AS
    acc_name varchar2(30);
BEGIN
    select name into acc_name from
    account where acc_no=account_num;
RETURN acc_name;
END;
```

**The Following
Function will
fetch the name of
the account for a
given account
number.**

2 Ways to Execute the Function:

```
DECLARE
    acc_name VARCHAR2(30);
BEGIN
    acc_name := account_det(&account_num);
    DBMS_OUTPUT.PUT_LINE(acc_name);
END;
```

```
Select
account_det('A001')
from account;
```

× ○ DIGITAL LEARNING CONTENT



Parul[®] University



www.paruluniversity.ac.in